

Arabic Agent using Google ADK

Saied “ساعد”

Team members:

Hashim Alhasani

Abdullah Alqahtani

Table of content

Arabic Agent using Google ADK.....	1
Table of content.....	2
Abstract.....	3
High Level Architecture.....	4
Architecture Deepdive.....	6
Global folder.....	6
__init__ → global folder.....	6
Agent.py.....	6
Memory.py.....	6
get_supabase:.....	6
embed_text:.....	6
save_memory:.....	6
search_memory:.....	7
format_memories:.....	7
load_recent:.....	7
clear_memory:.....	7
Prompt.py.....	7
Rag folder.....	8
embedder.....	8
ocr_with_qianfan.....	8
pdf_to_images.....	8
extract_text.....	8
chunk_text.....	8
embed (RAG).....	8
uploader.py.....	8
retriever.py.....	9
Tools folder.....	9
__init__ → Tools.....	9
calculator_tool.py.....	9
hijri_tool.py.....	9
gregorian_to_hijri.....	9
hijri_to_gregorian.....	9
reminder_tool.py.....	9
set_reminder.....	10
list_reminders.....	10
email_tool.py.....	10
send_email.....	10
rag_tool.py.....	10
search_documents.....	10
memory_tool.py.....	10
recall_memory.....	10
show_chat_history.....	10

forget_everything.....	11
prayer_tool.py.....	11
_get_location_from_ip.....	11
get_prayer_times.....	11
search_tool.py.....	11
web_search.....	11
poem_tool.py.....	11
search_poems.....	11
unit_converter.py.....	12
convert_units.....	12
pdfmerge_tool.py.....	12
merge_pdfs.....	12
Database Architecture.....	12
documents table.....	12
chat_memory table.....	12
poems table.....	13
Deployment.....	13

Abstract

Agents are getting more and more popular in this time, major players in the industry are focusing on shipping agents for various applications, such as customer serves, logs scrapper, personal secretary, and much more applications.

Here we noticed that there is not much focus on Arabic agents with tools that helps the muslim community.

We introduce “ساعد” the name came from the arm “ساعد” and help which holds the same name, this agent has the ability to get prayer time on your current location, convert georgian to hijri dates, set up reminders, and more tools which all will be specified later in this report.

High Level Architecture

The agent has multiple layers of architecture from local to cloud services.

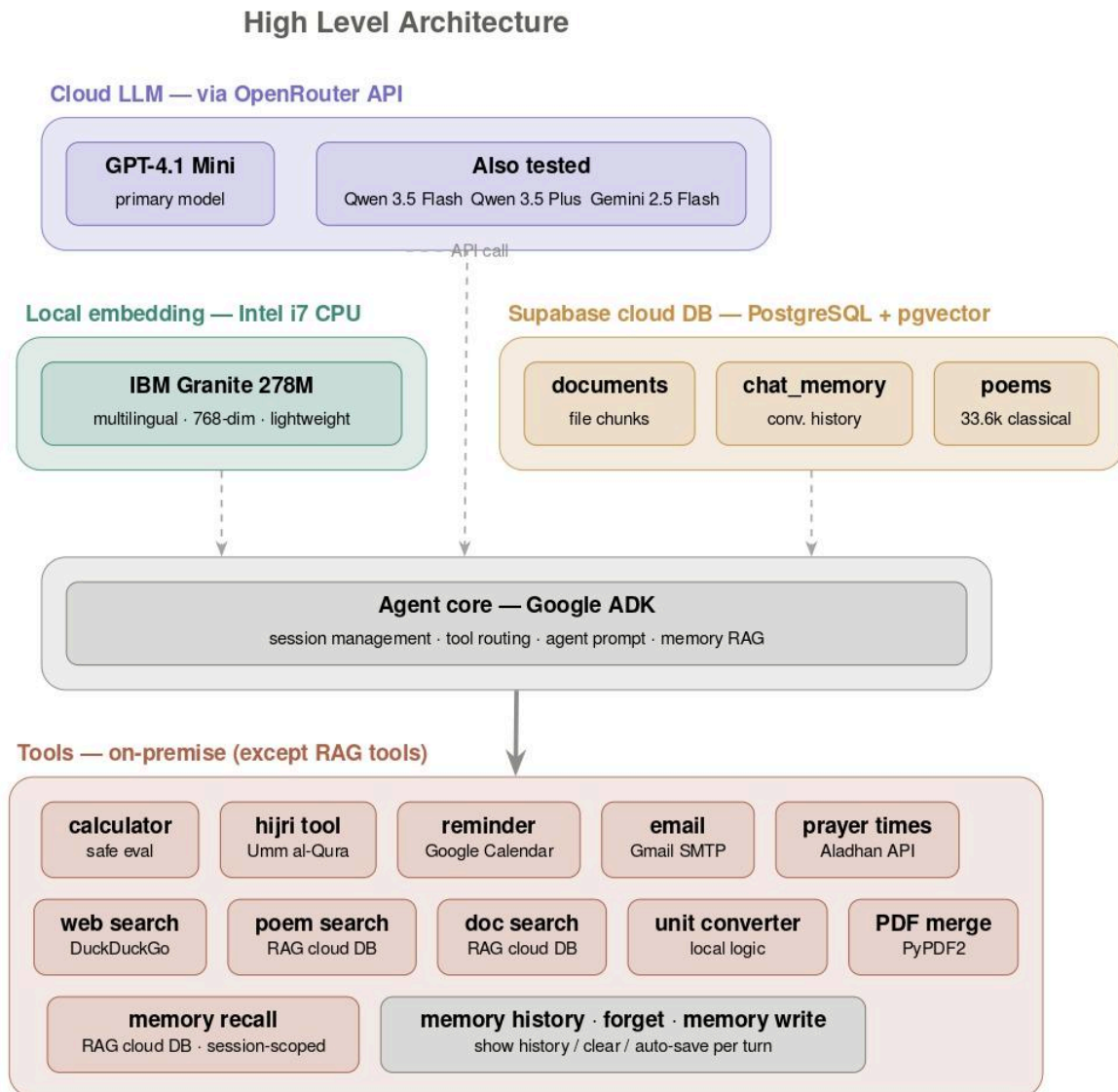
First the LLM which is cloud hosted and accessed using a paid API key, the provider is Openrouter and the model used is GPT 4.1 Mini, the reason we chose this model for the computing power it holds compared to token pricing, tool calling capability, and arabic understanding, this model checked all of the above requirements in testing, other models that was tested includes, Qwen 3.5 flash, Qwen 3.5 plus, Gemini 2.5 flash.

As for the embedding model it was locally hosted on a personal computer that has an intel i7 CPU with integrated GPU, the model used for embedding is ibm granite-embedding 278m multilingual, we chose this model for its capability for multilanguage understanding and its lightweight, this made this model a perfect fit for our use case.

The second which is the RAG system or the vector store, the vector store was hosted on a free of charge cloud database service which is Supabase postgres cloud database, this database holds all of the data needed for this project including, chat memory, document embeddings, and poem embeddings which will be explained further into this report.

The third is Tools, all of the tools are hosted on-premise, the only exception is the rag calling tool as explained earlier it calls for a cloud database.

Below is a figure describing the high level architecture for the project:



Architecture Deepdive

In this section I will explain all the architecture in details starting from the root agent until the last tool developed.

Global folder

`__init__` → global folder

This file's only function is to call the agent.

Agent.py

The agent architecture is provided by google ADK library, nothing was configured in case of temperature or repetition penalty, all is set to default.

The LLM used is GPT 4.1 mini with a paid API key provided by Openrouter

Memory.py

For the chat memory the library used is Google ADK for session management, sentence transformer for embedding model loading which as specified in the high level architecture section “see page 2” which is ibm granite-embedding 278m multilingual threading for locking embedding from overlapping in case the user was chatting frequently or in rapid succession.

below is the functions explained:

`get_supabase:`

this function is to create a client to connect to the cloud database, it loads the database using an API from the environment file. This function returns the Supabase client.

`embed_text:`

this function is used to embed the chat memory using the embedding model IBM Granite it uses the library sentence transformer to load and embed the content while locking the threading. This function returns the embeddings into a list.

`save_memory:`

this function is to upload the chatting into Supabase cloud database, it calls the functions `embed_text` to embed the chatting content and `get_supabase` to initialize the client and push the embeddings and the content into the cloud database, the reason we are embedding the chatting memory is to make the agent able to retrieve older memories on user command since the context window can be small for a mini model.

search_memory:

This function is used to query the cloud vector database for older memories in the chat, it takes session_id as filter for chats, query which the agent will instruct, and top_k which is defaulted to 5.

the function will call the embedding function to embed the query for a similarity search using cosine similarity, then calls the Supabase function to perform search on the cloud vector database, the function returns the results of the search in case of a success, and an error message in case of a failure.

format_memories:

This function is to format the retrieved memories from the vector database into readable structure to be loaded to the model.

this function returns the structured memory

load_recent:

This function is to load recent memories from the chat it takes the session id as a filter and the limit which is defaulted at 20, the function will call the get Supabase function to load an N number of chatting memory and in case of success it returns the memory and in case of failure it returns an exception.

clear_memory:

This function role is to clear the chatting memory it takes the session id as filter and calls get supabase function to execute a deletion command where the key searching value is the session id, in case of success it returns a conformation string and in case of failure it returns an exception.

Prompt.py

the prompt has no function, it only holds the formatted prompt string.

it uses the datetime library to inject the time into the prompt which will update every message.

Rag folder

embedder

this file uses these library to process and ingest documents, base64 for encoding each pdf file into a base64 image to scanned by OCR, fitz to extract word document content, langchain to use a recursive character splitter and memory from the memory file to use the same model and threading lock.

below is the functions explained:

ocr_with_qianfan

This function takes the base64 image to be scanned using a cloud based API provided by Openrouter

pdf_to_images

This function accepts a PDF file path and converts each page into a base64-encoded PNG image using PyMuPDF. The function returns a list of base64 strings, one per page.

extract_text

This function is the unified text extraction entry point. It accepts a file path and routes processing based on the file extension. For .txt files it reads directly. For .docx files it uses docx2txt. For .pdf files it always converts pages to images and runs OCR, since OCR is always enabled in this implementation to ensure consistent handling of both text-based and scanned documents. For image files (.png, .jpg, .jpeg) it encodes the image in base64 and runs OCR directly. The function returns the extracted text as a string.

chunk_text

This function splits extracted text into overlapping chunks using LangChain's RecursiveCharacterTextSplitter with a chunk size of 1500 characters and an overlap of 250 characters. The function returns a list of text chunks.

embed (RAG)

This function embeds a list of text chunks in a single batch using the shared IBM Granite model from memory.py. The threading lock is applied for the entire batch. The function returns a list of 768-dimensional embedding vectors, one per chunk.

uploader.py

This file is the full RAG ingestion pipeline. It uses Supabase client similar to memory.py. The upload_file function implements a four-step pipeline: first it extracts text from the file using the embedder, then chunks the text, then embeds all chunks in a single batch, and finally inserts all rows into the Supabase documents table. Each row contains the filename, the chunk text, and the embedding vector. The function returns a confirmation message on success.

retriever.py

This file handles document retrieval from the Supabase vector database. The retrieve function accepts a query string and a top_k parameter defaulting to 20. It embeds the query using the same IBM Granite model, then calls the Supabase match_documents RPC function which performs a cosine similarity search and returns the top_k most similar document chunks. The function returns a list of text strings.

Tools folder

`__init__` → Tools

All tools are registered in `__init__` through the `all_tools` list. This design choice was made to make collaboration easier so that `agent.py` never needs to be modified when a new tool is added. Each tool function includes an Arabic docstring that the LLM reads to determine when and how to call the tool. Below is a detailed description of each tool.

calculator_tool.py

This tool performs calculations. It accepts a mathematical expression as a string and evaluates it using Python's built-in `eval` function. Before evaluation, the input is validated against an allowed character set containing only digits, arithmetic operators, parentheses, and decimal points to prevent code injection. The function handles division by zero error and catches all other exceptions. It returns the result formatted as a string on success and an error message on failure.

hijri_tool.py

This file provides two date conversion functions using the `hijridate` library, which implements the Umm al-Qura calendar standard.

`gregorian_to_hijri`

This function accepts a date string in YYYY-MM-DD format and converts it from the Gregorian calendar to the Hijri calendar using the `hijridate` library's `Gregorian` class. The function validates the Gregorian year range (1924–2077) before conversion and returns the Hijri date as a formatted string on success.

`hijri_to_gregorian`

This function accepts a Hijri date string in YYYY-MM-DD format and converts it to the Gregorian calendar using the `hijridate` library's `Hijri` class. It follows the same single-string parameter design. The function validates the Hijri year range (1318–1500 Hijri) and returns the Gregorian date as a formatted string on success.

reminder_tool.py

This file provides Google Calendar integration using the Google Calendar Simple API (`gcsa`) library. Authentication is handled via OAuth 2.0. A token file is saved after the first authentication and reused on subsequent runs.

set_reminder

This function creates a Google Calendar event with an email notification. It accepts a title, date string in YYYY-MM-DD format, time string in HH:MM format, an optional note, and an optional `reminder_minutes` parameter defaulting to 30. The function validates that the scheduled time is in the future, creates a one-hour event using the `gcsa` Event class with an `EmailReminder`, and submits it to Google Calendar. The function returns a confirmation message on success.

list_reminders

This function retrieves upcoming calendar events from Google Calendar for the next 30 days. It calls the `GoogleCalendar` client to fetch events within that date range and formats them into a readable Arabic list with title, date, time, and optional description. The function returns a string listing all upcoming events, or a message indicating no events were found.

email_tool.py

send_email

This function sends emails via Gmail using Python's built-in `smtplib` library with SMTP over TLS on port 587. It accepts a recipient email address, subject, and body. The sender Gmail address and App Password are loaded from the environment file. The function constructs an `EmailMessage` object, authenticates via SMTP, and sends the message. The function returns a confirmation message on success and an error message on failure.

rag_tool.py

search_documents

This function is the tool the agent calls to access the RAG system. It calls the `retrieve` function from `retriever.py` with the user's query, joins the returned text chunks with separators, and returns them formatted as an Arabic string. The function returns an apology message if no results are returned.

memory_tool.py

This file provides three memory functions for the agent to access that wrap the `memory.py` module.

recall_memory

This function performs a semantic search over past conversation turns. It calls `search_memory` from `memory.py` with the query and `session_id`, then passes the results to `format_memories` to produce a readable context block. The function is used when the user references something said in a previous session or asks what was discussed before.

show_chat_history

This function retrieves and formats the last 20 messages from the conversation in chronological order by calling `load_recent` from `memory.py`. It is used when the user wants to review the recent conversation rather than searching semantically.

forget_everything

This function deletes all stored conversation memory for the current session by calling `clear_memory` from `memory.py`. The operation is irreversible. It is used when the user explicitly requests a memory reset.

prayer_tool.py

`_get_location_from_ip`

This is a private helper function that detects the user's city and country from their current IP address using the `ip-api.com` free geolocation API. It requests only the status, city, and country fields to minimize response size. If detection fails for any reason, it falls back to Riyadh, Saudi Arabia. The function returns a tuple of (city, country).

`get_prayer_times`

This function fetches daily prayer times for a given city using the Aladhan API with method 4, which corresponds to the Umm al-Qura University calculation method. Both city and country parameters are optional, if not provided, they are auto-detected using `_get_location_from_ip`. The function retrieves daily prayer times plus sunrise, along with the current Hijri and Gregorian dates from the API response. It formats and returns all information as a structured string. Timeout and connection errors are handled.

search_tool.py

`web_search`

This function performs web searches using the DuckDuckGo search engine via the `duckduckgo-search` library. It accepts a query string and a `max_results` parameter defaulting to 20. The function uses the DDGS context manager to execute a text search and formats the results with title, body, and URL for each result. It returns a formatted string containing all results, or a not-found message if no results are returned.

poem_tool.py

`search_poems`

This function performs semantic search over the Arabic poetry database stored in the Supabase poems table. The database contains 33,600 قصيد (Classical Arabic) poems from the `arbml/ashaar` HuggingFace dataset, filtered from an original 255,722 poems. The dataset was embedded on a GPU instance in Google Colab using the same IBM Granite model. The function accepts a query string, an optional theme filter, an optional poet era filter, and `top_k` defaulting to 20. It calls the Supabase `match_poems` RPC function, applies the optional filters in Python, and formats the results with title, poet name, era, meter, theme, and the first six verses. The function is used by the agent when the user writes a poem or requests poetry, it returns the poems as is and instructed to never change or create new ones.

unit_converter.py

convert_units

This function converts a numerical value between supported unit pairs. It accepts a float value, a source unit string, and a target unit string. Supported conversions are kilometres to miles and the reverse, kilograms to pounds and the reverse, and Celsius to Fahrenheit and the reverse. The conversion factors are stored in a dictionary keyed by unit pair tuples. The function performs a lookup and applies the corresponding conversion formula. It returns the rounded result as a string on success and an error message if the conversion pair is not supported.

pdfmerge_tool.py

merge_pdfs

This function merges multiple PDF files into a single output file using the PyPDF2 library's PdfMerger class. It accepts an ordered list of PDF file paths and an output file path. The function iterates over the input files, appending each to the merger, then writes the combined PDF to the specified output path. The merger is explicitly closed after writing to release file handles. The function returns a confirmation string with the output path on success and an error message on failure

Database Architecture

The Supabase PostgreSQL database uses the pgvector extension to store and query 768-dimensional embedding vectors. All three tables use the same vector dimension to match the IBM Granite embedding model output. Vector similarity search is performed using cosine similarity.

documents table

Stores text chunks from uploaded files alongside their 768-dimensional embeddings. Each row contains an auto-generated UUID, the source filename, the chunk text content, the embedding vector, and a creation timestamp. This table is written to by uploader.py and queried by retriever.py via the match_documents RPC function.

chat_memory table

Stores conversation turns alongside their embeddings for the Memory RAG system. Each row contains an auto-generated UUID, a session_id for conversation isolation, the role (user or assistant), the text content, the 768-dimensional embedding vector, and a creation timestamp. An additional composite index on session_id and created_at enables efficient chronological retrieval. This table is written to and queried by memory.py.

poems table

Stores Classical Arabic (فصيح) poems from the arbml/ashaar dataset. The original dataset contained 255,722 poems across five language type categories. all other categories were removed, and the remaining فصيح poems were further reduced to 33,600 rows through stratified random sampling by theme to fit within a 500MB storage budget. Each row stores the poem title, the full verse text, the meter (بحر), the theme, the poet name, the poet era, the poet location, the language type, and the 768-dimensional embedding vector. The dataset was embedded on a GPU in Google Colab using the same IBM Granite model.

Deployment

The system requires three terminals running simultaneously from the Agent_project root directory.

- Terminal 1: ADK api_server— `adk api_server "<agents_dir>" --allow_origins=""`
- Terminal 2: Flask upload server — `python upload_server.py`
- Terminal 3: HTTP server for frontend — `python -m http.server 3000`

The IBM Granite embedding model is loaded once at agent startup and kept in memory for the duration of the session. A threading lock ensures that concurrent embedding requests from memory writes and RAG queries do not conflict. The model is cached locally by Sentence Transformers after the first download and does not require internet access on subsequent startups